DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

IIIT Kottayam

TITLE OF THE PROJECT

SECURE MOBILE COMMUNICATION FRAMEWORK

Using Hybrid Cryptographic Techniques

PROJECT REPORT

Course Code: CSS411
Course Name: Cryptography and Network Security

Submitted by

Student Name	Roll No.
M. Kavya Amrutha	2023BCS0221
Harissh Ragav Dhamodaran	2023BCD0059
A. Pranathi	2023BCD0065
K.Neharika	2023BCD0053

Submitted to

Department of Computer Science and Engineering
IIIT Kottayam

2026

Title of the Project

Secure Mobile Communication Framework

Using Hybrid Cryptographic Techniques

Abstract

This project presents a Python-based Secure Mobile Communication Framework that combines complementary cryptographic techniques to support confidentiality, integrity, authentication, secure key exchange, and protection against common communication attacks. The framework uses Elliptic Curve Diffie–Hellman (ECDH) over the SECP256R1 curve for shared-secret establishment, AES-256 in Galois/Counter Mode (AES-256-GCM) for authenticated encryption, and RSA-2048 digital signatures for message authenticity.

User registration and login are supported through salted Password-Based Key Derivation Function 2 (PBKDF2) hashing with SHA-256, rather than plaintext password storage. A PySide6 graphical dashboard provides access to secure communication, cryptographic information, attack simulation, and activity logs. Demonstrations of eavesdropping, tampering, and replay attacks illustrate the different roles of encryption, integrity verification, and packet-freshness checks.

The implementation follows a modular client–server design using Python and TCP sockets. This report describes the proposed architecture, communication workflow, implementation responsibilities, and test plan. The framework is an educational prototype: production deployment would require stronger identity and key-management controls, additional protocol hardening, and independent security review. Supplied architecture diagrams illustrate the design, and GUI screenshots document the interface and displayed attack results. Independently verified test outcomes remain to be recorded before submission.

Keywords: Hybrid cryptography; secure communication; ECDH; AES-GCM; RSA signatures; PBKDF2; replay protection; PySide6.

Report scope. The project targets a mobile-communication use case, while the described implementation is a Python/PySide6 client–server prototype. Native mobile deployment is identified as future work.

Contents

Abstract **i**

1 Introduction **1**
1.1 Background . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 1
1.4 Existing and Proposed Systems . . . 1

2 Methodology **2**
2.1 Design Approach . . . 2
2.2 System Requirements . . . 2
2.3 System Architecture . . . 3
2.4 Authentication and Registration . . . 4
2.5 Cryptographic Techniques . . . 5
2.6 Secure Communication Workflow . . . 6
2.7 Attack Simulation . . . 7
2.8 Security Analysis . . . 8
2.9 Advantages and Limitations . . . 8
2.10 Future Scope . . . 8
2.11 Conclusion . . . 9

3 Screenshots **10**
3.1 Login and Registration . . . 10
3.2 Main Security Dashboard . . . 11
3.3 Secure Chat . . . 12
3.4 Cryptographic Keys . . . 13
3.5 Attack Simulator . . . 14
3.6 Attack Demonstration Input Stages . . . 15
3.7 Eavesdropping Result . . . 16
3.8 Tampering Result . . . 17
3.9 Replay Result . . . 18
3.10 Activity Logs . . . 19

4 Code **20**
4.1 Implementation Organization . . . 20
4.2 Representative Algorithm: Account Verification . . . 20
4.3 Representative Algorithm: Protected Communication . . . 21
4.4 Project Execution . . . 21

5 References **22**

1 Introduction

1.1 Background

Modern networked applications exchange information over channels that may be exposed to interception, modification, impersonation, or message replay. A secure communication system therefore needs more than a single encryption algorithm. It must establish keys securely, protect confidentiality and integrity, authenticate messages, and identify previously processed packets.

Hybrid cryptography combines the capabilities of symmetric and asymmetric mechanisms. In this project, ECDH supports shared-secret establishment, AES-256-GCM protects communication data, RSA-2048 signatures support message authenticity, and salted PBKDF2 hashing protects stored passwords. These functions are integrated into a modular Python client–server framework with a PySide6 dashboard.

1.2 Problem Statement

Plain or weakly protected communication can expose sensitive information to attackers. Encryption alone does not resolve every security requirement: a system also needs authenticated key establishment, reliable identity handling, integrity validation, and replay protection. Manual security testing can make these distinctions difficult to demonstrate clearly. The project addresses this problem by combining complementary mechanisms with practical attack demonstrations.

1.3 Objectives

- Provide message confidentiality and integrity using AES-256-GCM.
- Establish a shared secret using ECDH on SECP256R1 and obtain a session key.
- Support message authenticity using RSA-2048 digital signatures.
- Protect stored account passwords using salted PBKDF2-SHA256 hashing.
- Detect modifications to protected data and reject replayed packets.
- Demonstrate eavesdropping, tampering, and replay through a user-friendly dashboard.

1.4 Existing and Proposed Systems

A basic communication system may transmit information without strong confidentiality or integrity controls. Even a system using one encryption mechanism may leave gaps in authentication, key establishment, and replay protection.

The proposed framework integrates account authentication, ECDH, AES-GCM, RSA signatures, replay-detection logic, and an attack simulator. The graphical interface brings these operations into one environment, making the communication workflow and security responses easier to observe.

2 Methodology

2.1 Design Approach

The framework is organized around distinct responsibilities: account verification, cryptographic processing, network communication, attack simulation, and graphical interaction. Separating these responsibilities supports clearer implementation, testing, and future extension.

Table 1: Security requirements and corresponding mechanisms.

Requirement	Mechanism
User authentication	Username/password verification with salted PBKDF2 hashing
Shared-secret establishment	ECDH using SECP256R1
Confidentiality	AES-256-GCM encryption
Integrity	AES-GCM authentication tag verification
Message authenticity	RSA-2048 digital signature verification
Replay protection	Packet-freshness and replay-detection logic
Attack demonstration	Eavesdropping, tampering, and replay simulation
User interaction	PySide6 security dashboard

2.2 System Requirements

2.2.1 Hardware

A laptop or desktop computer is required, with at least 4 GB RAM recommended and a network interface for TCP communication. The framework may be demonstrated locally using separate client and server processes.

2.2.2 Software

Table 2: Software environment.

Software / Library	Purpose
Python 3	Application development and execution
PySide6	Graphical user interface
`cryptography`	Cryptographic primitives
TCP sockets	Client–server communication
Git / GitHub	Version control

2.3 System Architecture

The user interacts with the login or registration interface and then accesses the PySide6 dashboard. The client prepares protected packets and transmits them to the server over TCP. Reusable cryptographic functions provide the operations needed by the communication components.

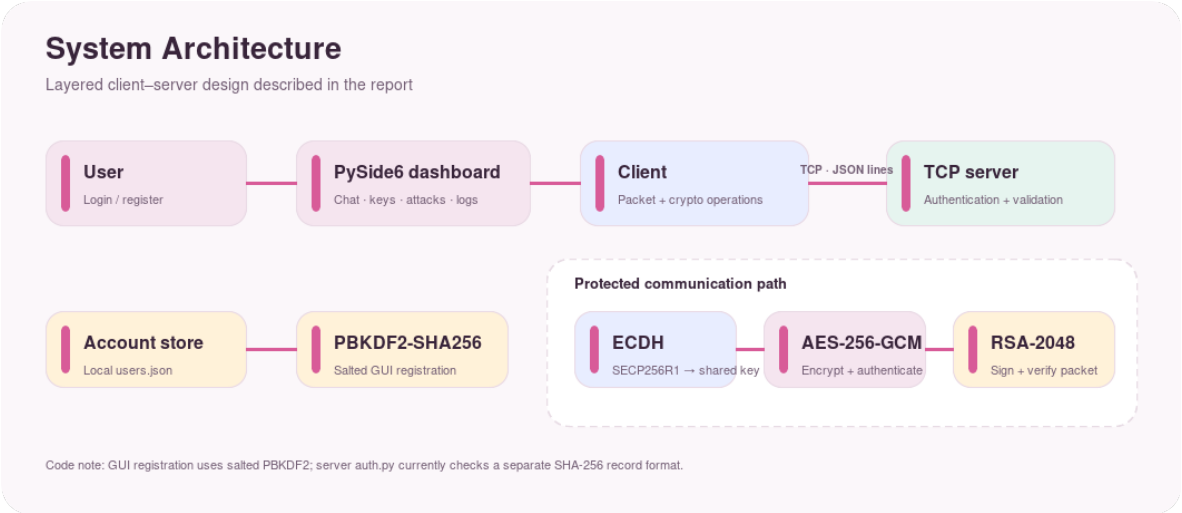


Figure 1: System architecture: the PySide6 interface, client–server communication, account storage, and cryptographic components.

2.4 Authentication and Registration

During registration, the user provides a username, password, and password confirmation. The application checks for duplicate usernames and mismatched passwords. A randomly generated salt is used with PBKDF2-SHA256 to derive a password hash. The salt and derived value are stored instead of the plaintext password.

During login, the entered password is processed using the stored salt and the configured derivation parameters. The resulting value is compared with the stored hash. Invalid usernames or passwords are rejected; successful verification opens the dashboard.

The uploaded registration diagram (Figure 2) describes a 16-byte random salt and 100,000 PBKDF2-HMAC-SHA256 iterations for the GUI account workflow. These settings are reported from the supplied asset and have not been independently checked against source code.

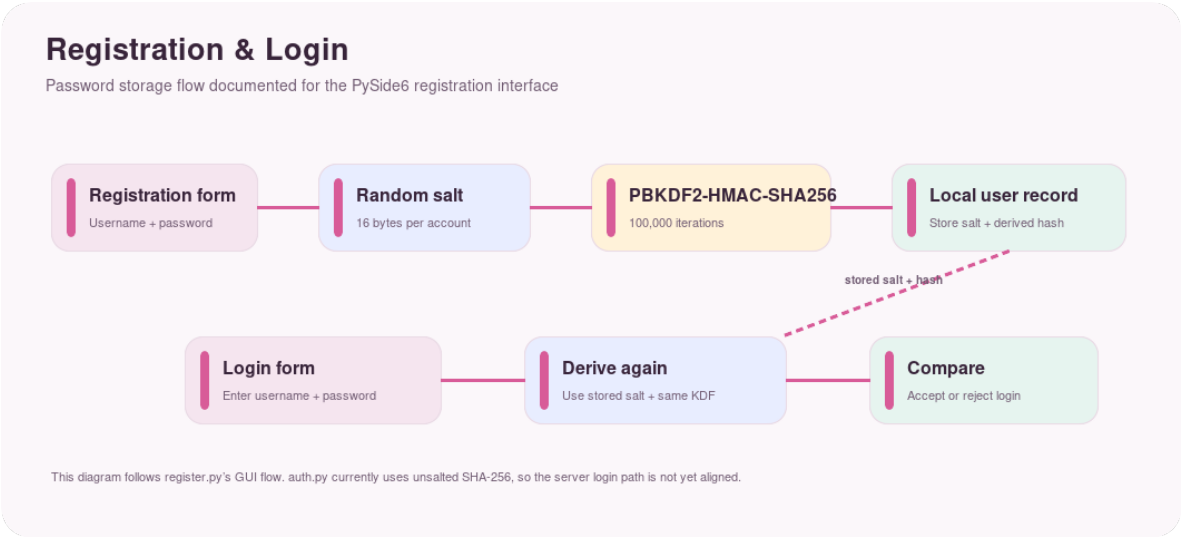


Figure 2: Registration and login workflow showing salted password derivation, local account storage, and credential verification.

Authentication integration caveat. The uploaded architecture and registration diagrams identify a separate server authentication path in `auth.py` that uses unsalted SHA-256 records. The GUI's salted PBKDF2 workflow must not be assumed to protect this separate path; the two account formats require alignment.

2.5 Cryptographic Techniques

2.5.1 AES-256-GCM

AES-256-GCM provides authenticated encryption of communication messages. The receiver verifies the authentication tag before accepting the recovered plaintext. Modification of protected ciphertext or authenticated metadata should cause verification to fail. A unique nonce is required for each encryption under the same key; nonce management is therefore an essential implementation requirement [1, 2].

2.5.2 ECDH with SECP256R1

Each communicating party generates an elliptic-curve key pair and exchanges its public key while keeping its private key local. ECDH allows both parties to compute a shared secret. The shared secret should be processed through a suitable key-derivation function to obtain the AES session key; HKDF is a relevant design option [5]. The uploaded workflow diagram identifies HKDF-SHA256 for session-key derivation (Figure 3); its detailed parameters still require source-code verification.

2.5.3 RSA-2048 Digital Signatures

RSA signatures provide a mechanism for checking the authenticity and integrity of signed data. The sender signs using a private key, and the receiver verifies using the corresponding public key. This assurance depends on trusting the association between the public key and its owner. The uploaded workflow diagram specifies RSA-PSS with SHA-256 (Figure 3); the exact signed packet fields still require source-code verification [3].

2.5.4 PBKDF2 Password Hashing

PBKDF2-SHA256 uses a randomly generated salt and a configured iteration count to derive a stored password value. Login verification repeats the derivation using the stored parameters. Password hashing protects account records but is separate from message encryption and key exchange [4].

2.6 Secure Communication Workflow

1. **Authenticate the user.** Verify the registered account credentials.
2. **Establish the shared secret.** Exchange ECDH public keys and compute the shared secret.
3. **Obtain the session key.** Derive the key material required by AES-256-GCM.
4. **Protect the message.** Encrypt using AES-GCM with a fresh nonce and generate the RSA signature over the defined packet data.
5. **Transmit the packet.** Send the protected data to the server over TCP.
6. **Verify and accept.** Check packet format, freshness, signature, and authentication tag before accepting plaintext. Record accepted packet identifiers to prevent reuse.

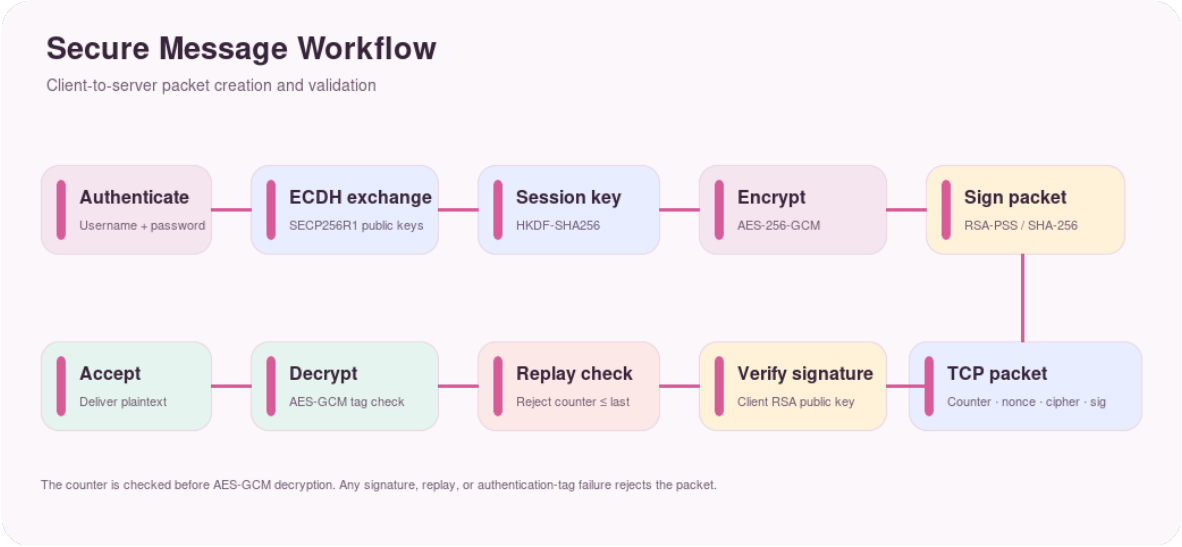


Figure 3: Secure-message workflow: ECDH key exchange, session-key derivation, encryption, signing, TCP transmission, and packet verification.

The supplied workflow identifies a counter-based replay check that rejects counters less than or equal to the last accepted value before AES-GCM decryption. This is a design description, not an independently verified execution trace.

Protocol boundary. ECDH alone does not authenticate the peer, and AES-GCM alone does not reject a replay of an otherwise valid packet. Peer-key trust and explicit replay checks must be designed alongside these primitives.

2.7 Attack Simulation

The simulator demonstrates three communication threats. Eavesdropping observes encrypted traffic; tampering modifies protected data; replay resends a previously captured valid packet. Figure 4 illustrates these scenarios, and the table summarizes their expected responses. The corresponding interface captures appear in Figures 14–16; displayed statuses are distinguished from independently verified security outcomes.

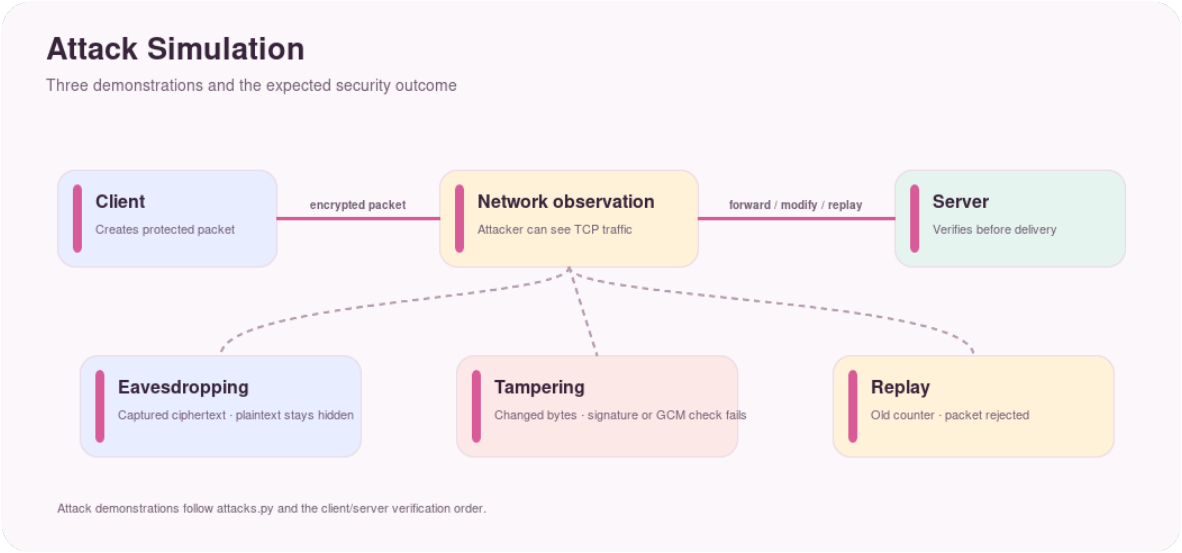


Figure 4: Attack-simulation architecture showing the client, network observation, server, and expected responses to eavesdropping, tampering, and replay.

Table 3: Attack scenarios and expected security responses.

Attack	Action	Expected Response
Eavesdropping	Observe encrypted traffic	Plaintext is not directly exposed
Tampering	Modify a protected packet	Integrity or signature verification fails
Replay	Resend a captured packet	Previously accepted packet is rejected

2.8 Security Analysis

The framework demonstrates defense in depth by assigning different responsibilities to different mechanisms. ECDH supports shared-secret establishment, AES-GCM protects confidentiality and integrity, RSA signatures support authenticity, and PBKDF2 protects stored passwords. Explicit freshness checks address replay attacks.

Production assurance additionally depends on authenticated peer keys, nonce uniqueness, secure private-key storage, defined packet serialization, and robust error handling. These controls require verification against the actual source code; the choice of cryptographic algorithms alone does not establish complete protocol security.

2.9 Advantages and Limitations

2.9.1 Advantages

- Combines symmetric and asymmetric cryptographic techniques in one workflow.
- Uses authenticated encryption and supports signed message verification.
- Establishes shared secrets using ECDH and avoids plaintext password storage.
- Includes practical attack simulations and a visual security dashboard.
- Uses a modular implementation that can support further development.

2.9.2 Limitations

- The framework is an educational/local prototype, not an audited production protocol.
- Production-grade identity verification, certificate management, and persistent key protection are limited or require further design.
- Public-network deployment requires additional hardening and independent security review.
- Native mobile deployment and multi-client scaling remain future extensions.
- Interface screenshots are supplied, but source-verified implementation parameters and independently executed test results remain unavailable.

2.10 Future Scope

- Introduce secure persistent key management and session-key rotation.
- Add certificate-based peer identity verification.
- Support multiple concurrent clients and stronger server-side account management.
- Extend the framework to mobile platforms.
- Add richer security analytics, visualization, and automated security tests.
- Perform formal protocol analysis and independent security review.

2.11 Conclusion

The Secure Mobile Communication Framework demonstrates how hybrid cryptographic mechanisms can be combined into a layered communication workflow. ECDH supports shared-secret establishment, AES-256-GCM protects message confidentiality and integrity, RSA-2048 signatures support authenticity, and salted PBKDF2 hashing protects account passwords.

The graphical interface and attack simulator provide an educational setting for examining eavesdropping, tampering, and replay threats. The modular design offers a foundation for stronger identity management, key rotation, mobile deployment, and formal security evaluation. Its effectiveness must ultimately be documented through the final implementation and reproducible test evidence.

3 Screenshots

The supplied GUI screenshots document the application's account screens, dashboard pages, and attack demonstrations. Architecture and workflow diagrams are presented separately in Methodology (Figures 1–4). The original asset filenames are retained, with each capture assigned according to its visible screen content.

3.1 Login and Registration

The login screen is the initial interface opened by `login.py`. Selecting **CREATE NEW ACCOUNT** opens the registration form, which requests a username, password, and password confirmation.

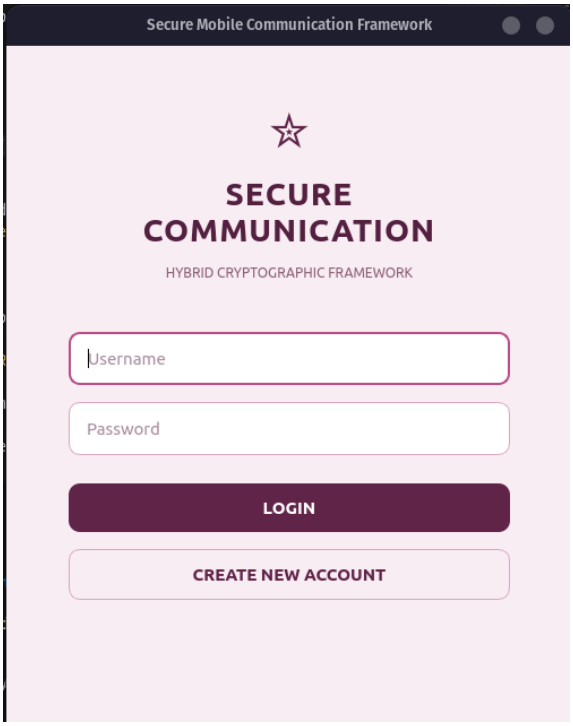


Figure 5: Login screen with account credentials and the registration shortcut.

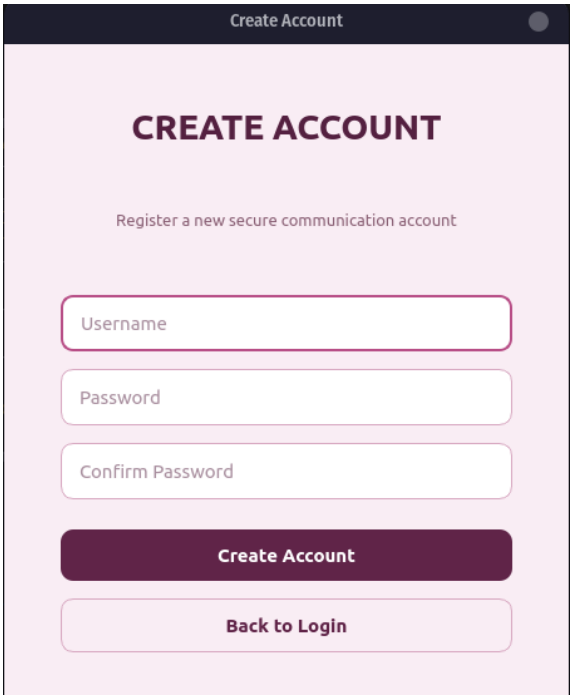


Figure 6: Registration screen with username, password, and confirmation fields.

3.2 Main Security Dashboard

After login with the demo account, the Security Dashboard provides navigation to Secure Chat, Cryptographic Keys, Attack Simulator, and Activity Logs. The supplied capture shows security-status indicators and recent activity.

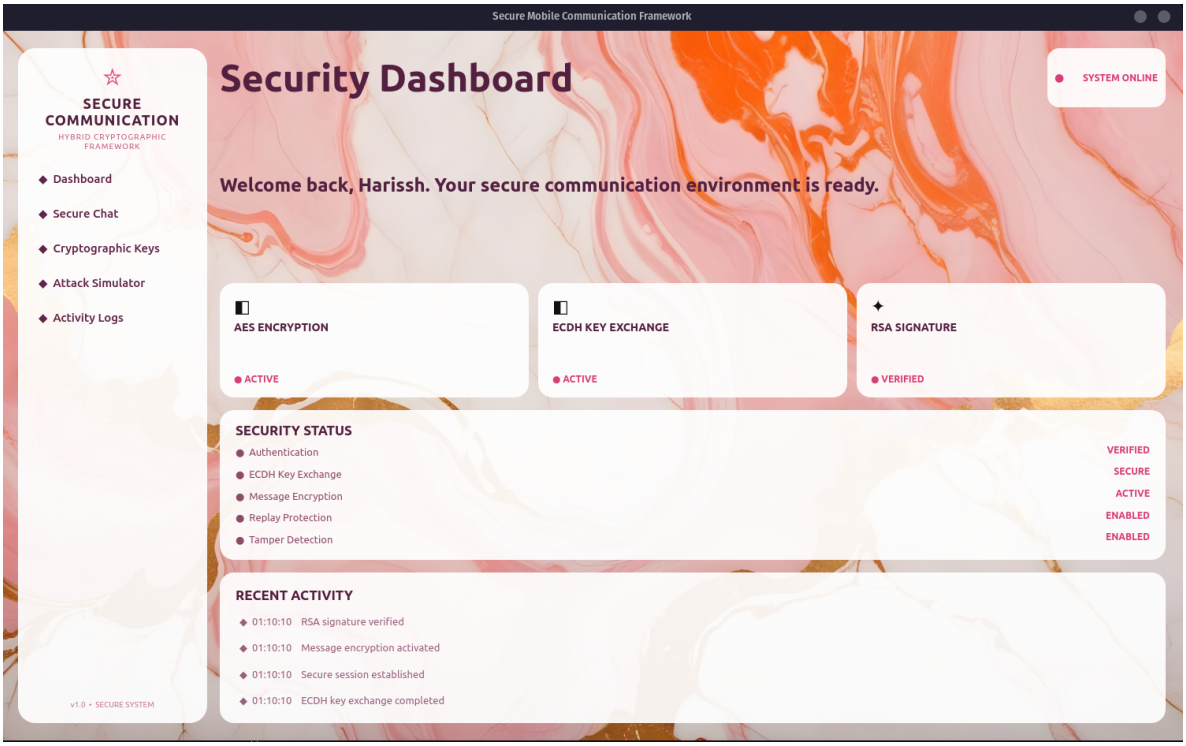


Figure 7: Main Security Dashboard showing cryptographic status, security indicators, recent activity, and page navigation.

3.3 Secure Chat

Selecting **Secure Chat** opens the message form. The supplied capture shows the account fields, a sample message, the **SEND SECURE MESSAGE** control, and a notification stating that the message was encrypted successfully. This notification alone does not establish successful server delivery.

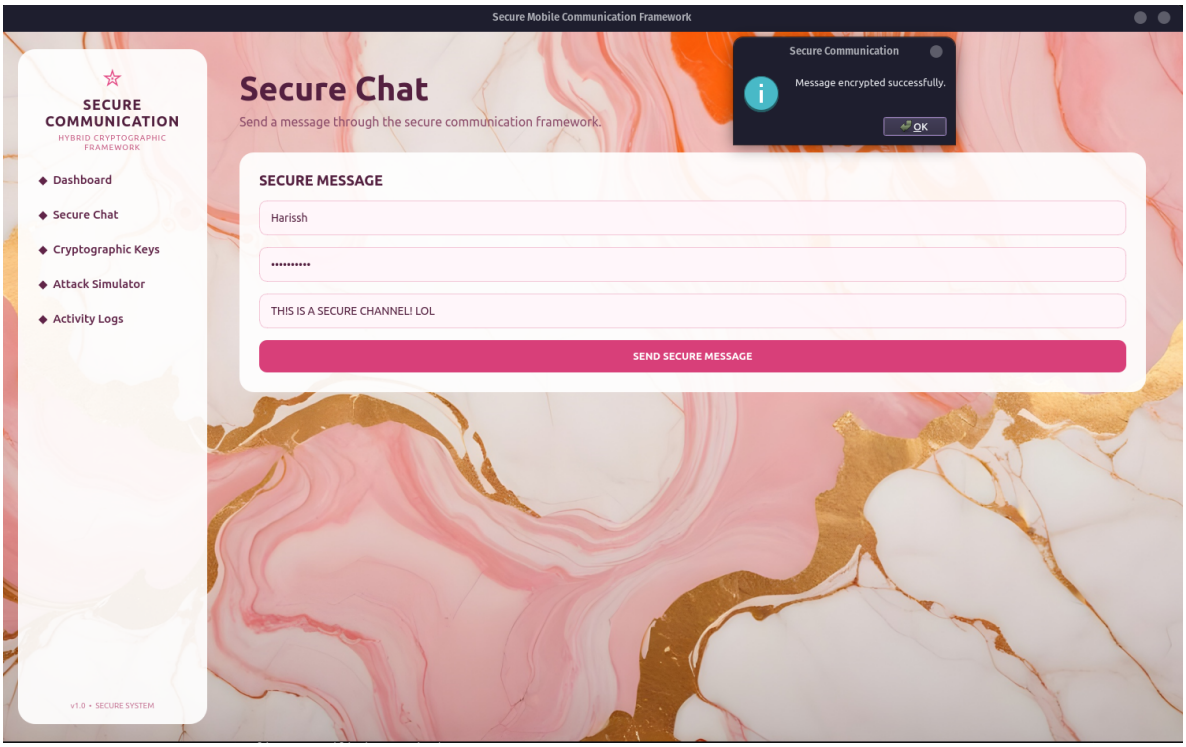


Figure 8: Secure Chat page with the message form and encryption-success notification.

3.4 Cryptographic Keys

Selecting **Cryptographic Keys** opens the overview of AES-256-GCM encryption, ECDH key exchange, and RSA-2048 signatures. The screenshot also includes an AES Encryption status dialog; it is an interface overview rather than a record of raw key material.

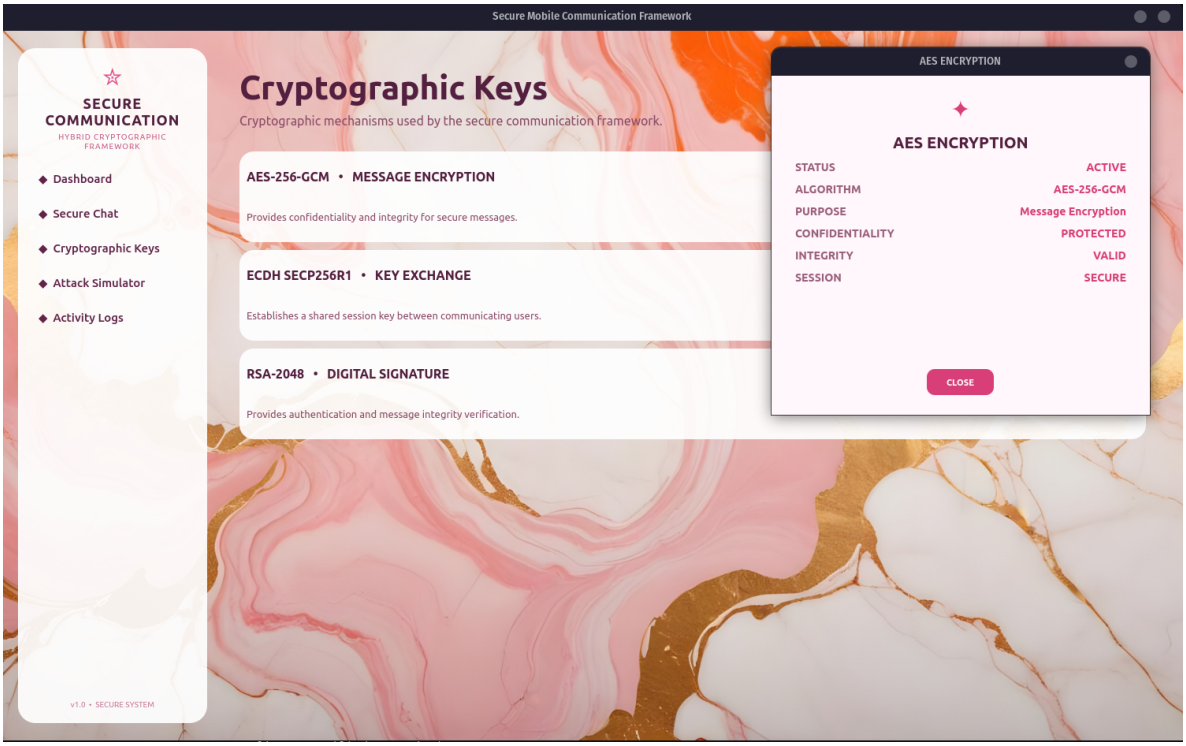


Figure 9: Cryptographic Keys page and AES Encryption status dialog.

3.5 Attack Simulator

Selecting **Attack Simulator** opens the three demonstration choices: **EAVESDROPPING**, **TAMPERING**, and **REPLAY ATTACK**. The supplied capture shows the simulator menu shared by all three attacks.

The attack demonstrations use TCP communication. Before reproducing the result captures, start the server in a separate terminal from the application project directory:

```
python server.py
```

Then select an attack and enter the demo account credentials and a sample message when prompted. The application source files are not included in this report workspace, so these steps were not executed here.



Figure 10: Attack Simulator menu with eavesdropping, tampering, and replay options.

3.6 Attack Demonstration Input Stages

The supplied eavesdropping-stage captures show the input sequence before the result window: username, password, and sample message. They supplement the simulator and final-result screenshots.

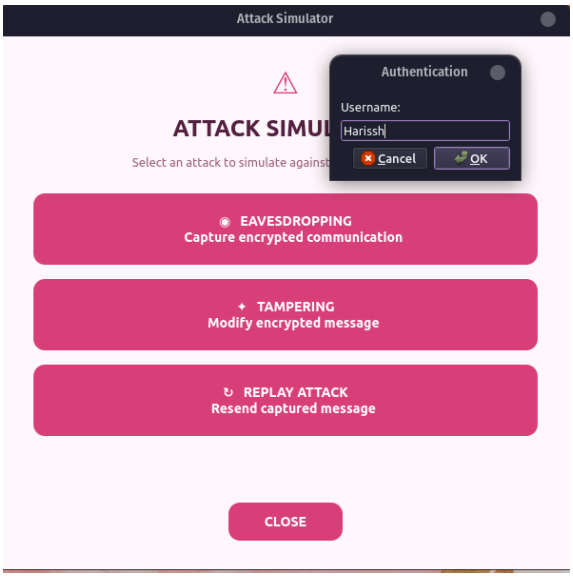


Figure 11: Attack input, stage 1: username prompt.

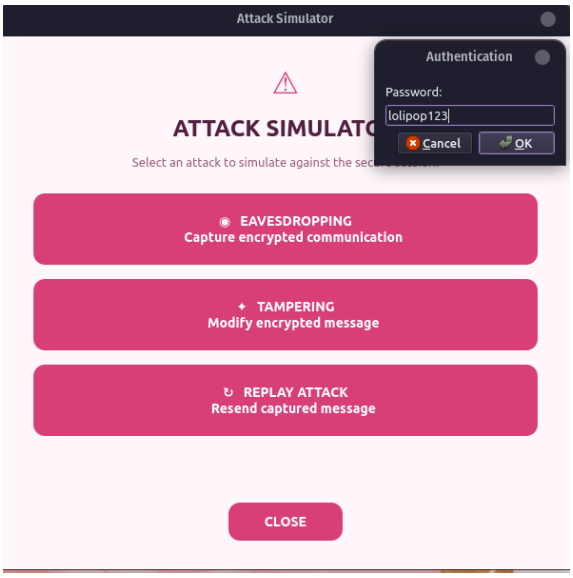


Figure 12: Attack input, stage 2: password prompt.



Figure 13: Attack input, stage 3: sample-message prompt.

3.7 Eavesdropping Result

The eavesdropping result displays captured ciphertext and a **PROTECTED** status. The window states that the attacker can see ciphertext but not the original plaintext; the screenshot records this displayed outcome rather than an independently inspected TCP capture.

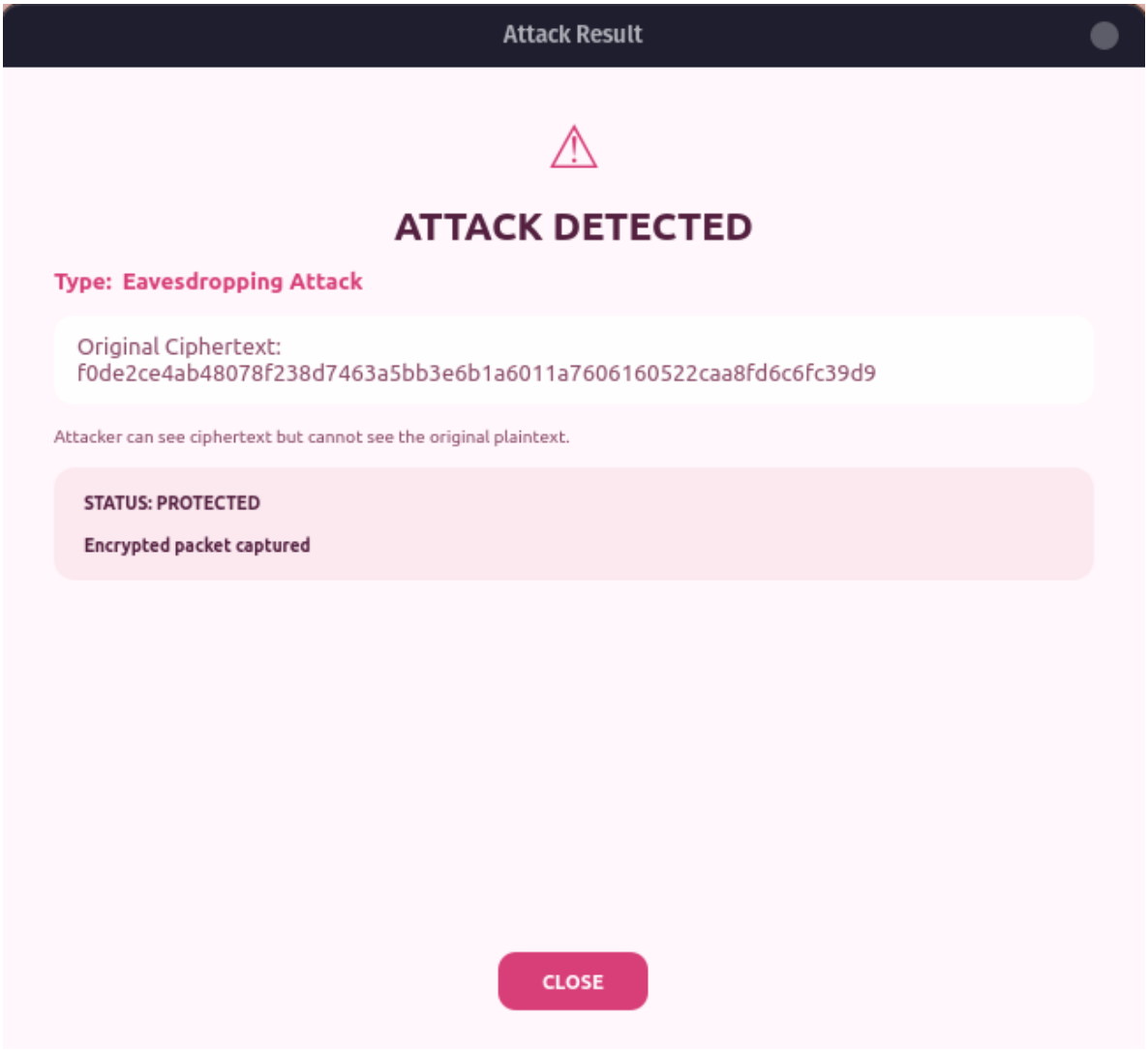


Figure 14: Eavesdropping result showing captured ciphertext and the displayed protected status.

3.8 Tampering Result

The tampering result displays the original and modified ciphertext and labels the attack as **DETECTED**. Its response contains `MESSAGE_REJECTED` with `KeyError`, so the capture alone does not establish that rejection was caused by a signature or authentication-tag check.

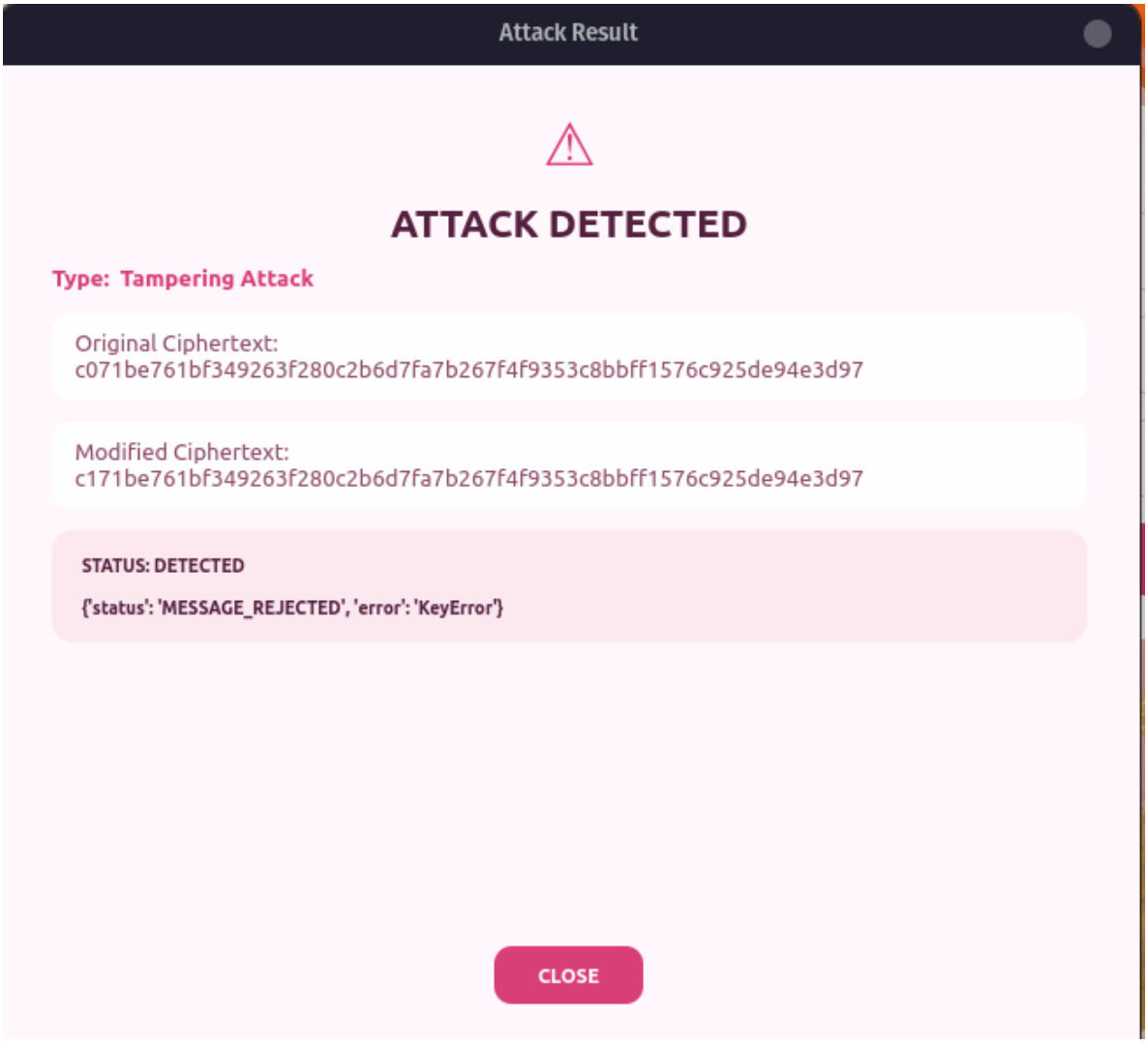


Figure 15: Tampering result showing original and modified ciphertext and a message-rejection response.

3.9 Replay Result

The replay result displays the captured ciphertext and labels the attack as **DETECTED**. The visible response also contains `MESSAGE_REJECTED` with `KeyError`; this does not by itself verify that a replay counter or freshness check rejected the packet.

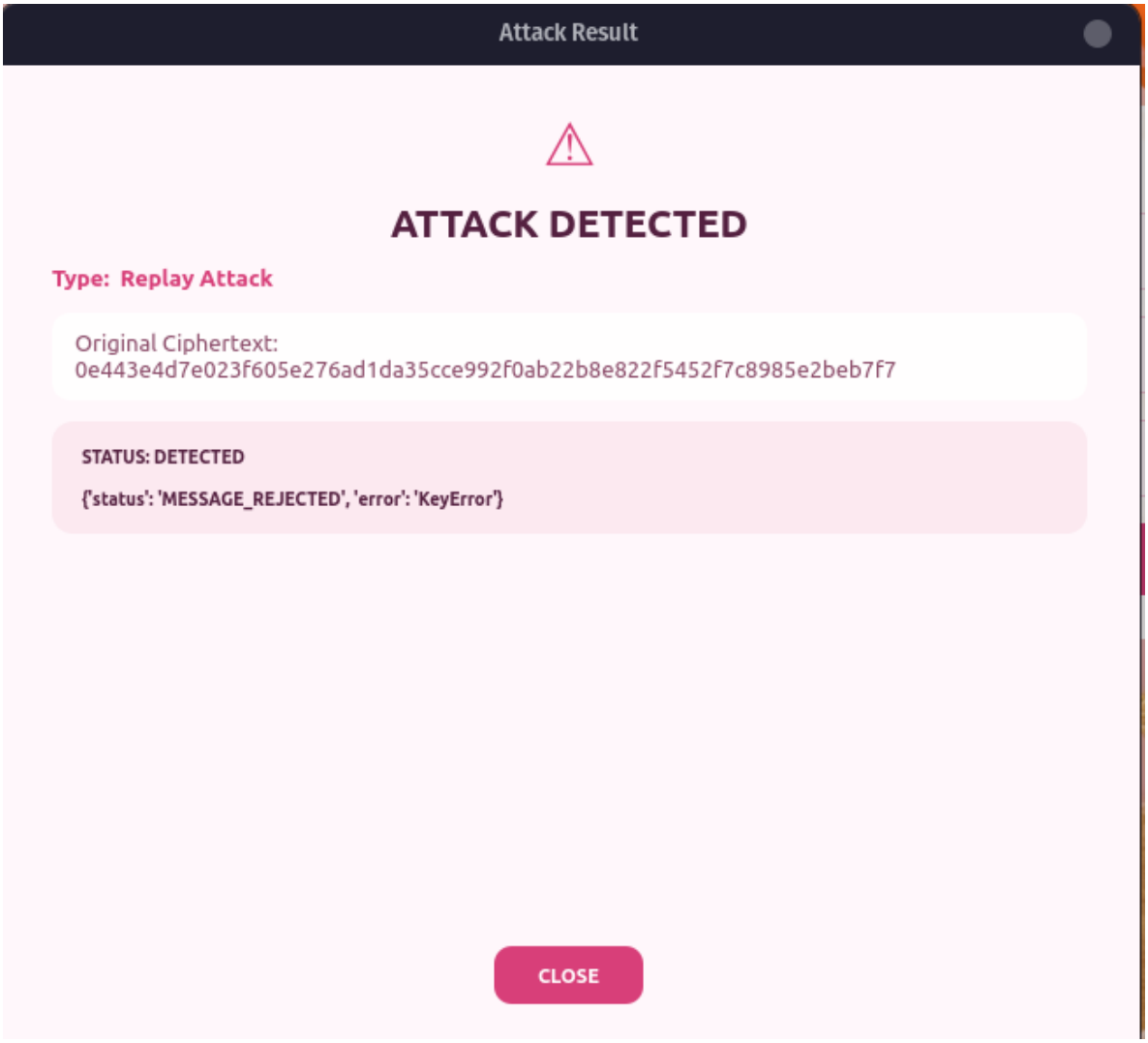


Figure 16: Replay-attack result showing captured ciphertext and a message-rejection response.

Result interpretation. The supplied attack screenshots document what the GUI displayed. The tampering and replay errors require investigation against the application code and server logs before either test can be recorded as passed.

3.10 Activity Logs

Selecting **Activity Logs** after the demonstrations displays the event history. The capture lists cryptographic operations, attack-simulation events, and communication failures; it is not a record of exclusively successful exchanges.

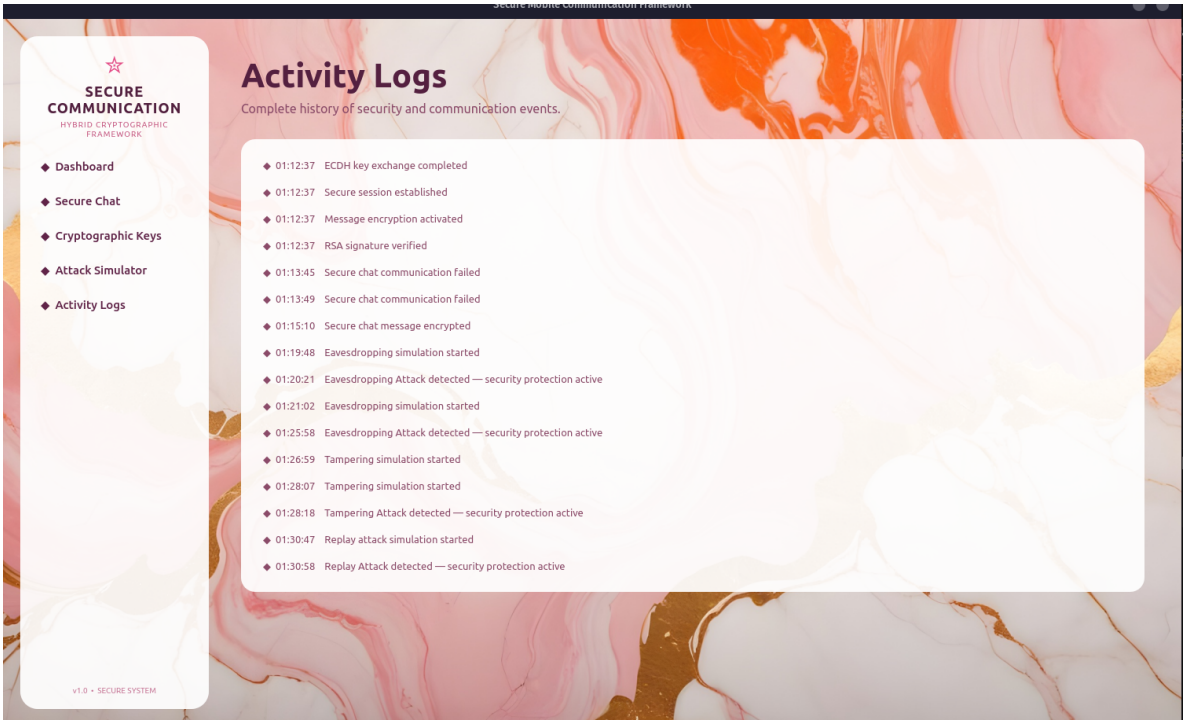


Figure 17: Activity Logs page showing cryptographic operations, communication events, and attack-demonstration history.

4 Code

4.1 Implementation Organization

The implementation is divided into interface, communication, cryptographic, and simulation modules. The supplied project description identifies the responsibilities listed below.

GitHub Repository:
https://github.com/Kats32/Secure-Mobile-Communication-Framework

Table 4: Project files and their responsibilities.

File	Responsibility
`login.py`	Login interface and password verification
`register.py`	Registration interface and password hashing
`dashboard.py`	Main dashboard and navigation
`client.py`	Client-side secure packet creation and communication
`server.py`	Server-side communication and security processing
`crypto_utils.py`	Reusable cryptographic helper functions
`attacks.py`	Eavesdropping, tampering, and replay demonstrations
`pages.py`	Dashboard sub-pages
`users.json`	Local user records; excluded from version control

4.2 Representative Algorithm: Account Verification

Listing 1: Pseudocode for registration and login.

```
REGISTER(username, password, confirmation):
    reject if username already exists
    reject if password differs from confirmation
    salt = RANDOM_BYTES(configured_salt_length)
    password_hash = PBKDF2_SHA256(
        password, salt, configured_iterations)
    STORE(username, salt, password_hash, parameters)

LOGIN(username, entered_password):
    record = LOOKUP_USER(username)
    reject if record does not exist
    candidate = PBKDF2_SHA256(
        entered_password, record.salt,
        record.parameters.iterations)
    accept only if CONSTANT_TIME_EQUAL(
        candidate, record.password_hash)
```

4.3 Representative Algorithm: Protected Communication

The workflow below illustrates the intended security responsibilities. Exact packet fields, signature configuration, key derivation, and replay-state management must be reconciled with the actual implementation.

Listing 2: Conceptual protected-packet workflow.

```
ESTABLISH_SESSION(local_private_key, peer_public_key):
    VERIFY_TRUSTED_PEER_KEY(peer_public_key)
    shared_secret = ECDH(local_private_key, peer_public_key)
    session_key = DERIVE_SESSION_KEY(shared_secret)

SEND_MESSAGE(message, session_key, signing_key):
    nonce = NEXT_UNIQUE_NONCE(session_key)
    metadata = NEW_SESSION_BOUND_PACKET_METADATA()
    ciphertext_and_tag = AES_256_GCM_ENCRYPT(
        session_key, nonce, message, metadata)
    signed_data = CANONICAL_ENCODE(
        metadata, nonce, ciphertext_and_tag)
    signature = RSA_SIGN(signing_key, signed_data)
    TRANSMIT(signed_data, signature)

RECEIVE_PACKET(packet, session_key, trusted_sender_key):
    VALIDATE_PACKET_FORMAT(packet)
    REJECT_IF_ALREADY_ACCEPTED_OR_STALE(packet.metadata)
    RSA_VERIFY(trusted_sender_key,
               packet.signed_data, packet.signature)
    plaintext = AES_256_GCM_DECRYPT_AND_VERIFY(
        session_key, packet.nonce,
        packet.ciphertext_and_tag, packet.metadata)
    ATOMICALLY_RECORD_ACCEPTED_PACKET(packet.metadata)
    DELIVER(plaintext)
```

4.4 Project Execution

The following commands reproduce the setup instructions supplied for the project. They are included for the final project environment and were not executed while preparing this report.

Listing 3: Application setup and launch commands.

```
python3 -m venv venv
source venv/bin/activate
pip install cryptography PySide6
python login.py
```

Start the server in a separate terminal when client–server communication is required:

```
python server.py
```

The activation command shown above assumes a Unix-like shell. Run commands from the application directory containing the listed Python modules.

5 References

[1] National Institute of Standards and Technology (NIST). *Advanced Encryption Standard (AES)*. FIPS PUB 197.

[2] National Institute of Standards and Technology (NIST). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D.

[3] National Institute of Standards and Technology (NIST). *Digital Signature Standard (DSS)*. FIPS PUB 186-5.

[4] National Institute of Standards and Technology (NIST). *Recommendation for Password-Based Key Derivation: Part 1: Storage Applications*. NIST Special Publication 800-132.

[5] H. Krawczyk and P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869.

[6] Python Cryptographic Authority. *Python Cryptography Documentation*.